

SIMATS ENGINEERING

ASSESSMENT TOOL 2: Problem Solving Assignment

COURSE NAME: ARTIFICIAL INTELLIGENCE COURSE CODE: MLA01
AND EXPERT SYSTEMS

COURSE OUTCOME COVERED

CO2: Analyze and select appropriate agent strategies for different types of environments and problem settings. (BTL 3)

Assessment Tool Weightage: 40 %

Total Marks: 100

Time: 60 Mins

No. of Questions: 5

Annexure A Problem Solving Assignment

Code of Conduct:

I Sk. Fawaz / 192425390 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Sk. Fawaz Ahamad

Signature of the student:

Criterion	Weightage	Marks Obtained
Problem Analysis and Understanding	15%	
AI Algorithm Selection & Justification	15%	
Algorithm Design / Pseudocode	25%	
Application of AI Concepts (Greedy, A*, CSP, Minimax, etc.)	15%	
Diagram / State Space / Search Tree Representation	10%	
Complexity Analysis and Solution Efficiency	10%	
Originality, Critical Thinking, and Presentation	10%	
Total	100%	

Signature of the Faculty

Total Marks Awarded

Answer All the Questions

Q.No	Application-Based Question	Major CO2 Concepts
1	Smart Ambulance Navigation: An ambulance must travel from a hospital to an emergency location through a city road network. Some roads have higher travel costs, and an estimated distance to the destination is available for each junction. Design a search-based solution to identify the best route. Using the given road network and heuristic values, determine the route selected by your algorithm, show the sequence of junctions explored, calculate the relevant evaluation values, and justify whether the route is optimal.	Informed Search, A*, Heuristic Function
2	Autonomous Warehouse Robot: A robot has to move from a charging station to a target shelf in a warehouse. The robot does not know all obstacles initially and discovers some obstacles only when it reaches nearby locations. Design an intelligent search strategy for the robot. Demonstrate the robot's decisions for the given grid, explain how it reacts when a new obstacle is discovered, and compare the approach with an offline search strategy.	Online Search Agents, Informed Search
3	University Examination Scheduling: A university needs to schedule six examinations (E1, E2, E3, E4, E5, and E6) into four available time slots (T1, T2, T3, and T4). Some examinations have common students and therefore cannot be conducted in the same time slot.	CSP, MRV, Forward Checking, Constraints

	The following constraints must be satisfied: 1. E1 and E2 have common students. 2. E1 and E3 have common students. 3. E2 and E4 have common students. 4. E3 and E5 have common students. 5. E4 and E6 have common students. 6. E2 and E5 have common students. The university has three examination halls with the following capacities: <table><tr><th>Examination Hall</th><th>Capacity</th></tr><tr><td>Hall A</td><td>120 students</td></tr><tr><td>Hall B</td><td>90 students</td></tr><tr><td>Hall C</td><td>60 students</td></tr></table> <table><tr><th>Examination</th><th>E1</th><th>E2</th><th>E3</th><th>E4</th><th>E5</th><th>E6</th></tr><tr><td>Number of Students</td><td>100</td><td>80</td><td>70</td><td>60</td><td>90</td><td>50</td></tr></table> Tasks: a) Formulate the problem as a Constraint Satisfaction Problem (CSP) by clearly defining the variables, domains, and constraints. b) Use Backtracking with the Minimum Remaining Values (MRV) heuristic and Forward Checking to obtain a valid examination timetable. c) Show the domain of each variable after every assignment and provide the final timetable, ensuring that all student-conflict and hall-capacity constraints are satisfied.	Examination Hall	Capacity	Hall A	120 students	Hall B	90 students	Hall C	60 students	Examination	E1	E2	E3	E4	E5	E6	Number of Students	100	80	70	60	90	50	
Examination Hall	Capacity																							
Hall A	120 students																							
Hall B	90 students																							
Hall C	60 students																							
Examination	E1	E2	E3	E4	E5	E6																		
Number of Students	100	80	70	60	90	50																		
4	Game-Playing AI: You are developing an AI opponent for a board game. The current game position produces the following possible moves and terminal scores in the supplied game tree. Determine the move the AI should make by applying Minimax. Then apply Alpha-Beta Pruning and identify the branches that do not need to be evaluated. Explain how pruning improves the efficiency of the game-playing agent.	Game Playing, Minimax, Alpha-Beta Pruning																						

	MAX MIN MAX 3 5 6 MAX 9 1 2 MAX 0 -1 4 MIN MAX 7 5 8 MAX 6 2 3 MAX 4 0 -2 MIN MAX 1 2 3 MAX 5 -1 2 MAX 6 4 7																																				
5	Delivery Route Optimization: A logistics company wants to minimize the total distance travelled by a delivery vehicle visiting multiple customers. An initial route and several neighboring routes are provided. Design an optimization strategy to improve the route. Apply the strategy step-by-step to the given routes, determine the final route, explain whether it represents a global optimum, and suggest how the algorithm could escape a local optimum. Distance matrix (in km) <table><tr><td></td><td>D</td><td>A</td><td>B</td><td>C</td><td>E</td></tr><tr><td>D</td><td>–</td><td>6</td><td>9</td><td>8</td><td>7</td></tr><tr><td>A</td><td>6</td><td>–</td><td>3</td><td>5</td><td>6</td></tr><tr><td>B</td><td>9</td><td>3</td><td>–</td><td>4</td><td>5</td></tr><tr><td>C</td><td>8</td><td>5</td><td>4</td><td>–</td><td>3</td></tr><tr><td>E</td><td>7</td><td>6</td><td>5</td><td>3</td><td>–</td></tr></table> Some neighboring routes of the initial solution and their costs S1: D → A → C → B → E → D = 26 km S2: D → B → A → C → E → D = 30 km S3: D → A → B → E → C → D = 24 km S4: D → E → C → B → A → D = 27 km S5: D → A → E → B → C → D = 23 km S6: D → C → B → A → E → D = 29 km From S5, the best neighbor has cost 22 km. From that solution, all neighbors have cost ≥ 22 km. Apply the chosen local search strategy and report the final route. Local Search, Optimization, Hill Climbing		D	A	B	C	E	D	–	6	9	8	7	A	6	–	3	5	6	B	9	3	–	4	5	C	8	5	4	–	3	E	7	6	5	3	–
	D	A	B	C	E																																
D	–	6	9	8	7																																
A	6	–	3	5	6																																
B	9	3	–	4	5																																
C	8	5	4	–	3																																
E	7	6	5	3	–																																

Q1. Smart Ambulance Navigation

1. Problem Analysis and Understanding

The ambulance must travel from the **hospital (S)** to the **emergency location (G)** through a city road network. Each road has a travel cost, while every junction has a heuristic value, representing the estimated remaining cost to the destination.

The objective is to find a route with the **minimum total travel cost**. Since both actual road costs and heuristic estimates are provided, **A* Search** is appropriate.

For A*:

$$f(n)=g(n)+h(n)$$

where:

- = actual cost from start to node
- = estimated cost from to goal
- = estimated total cost through

2. AI Algorithm Selection & Justification

Selected Algorithm: A* Search

A* is suitable because it combines the advantages of:

- **Uniform Cost Search** – considers actual road cost
- **Greedy Best-First Search** – considers estimated distance

Therefore, A* can efficiently find a low-cost/optimal route when the heuristic is appropriate.

A simple greedy search using only could choose a junction that looks close to the goal but has an expensive route. A* avoids this by considering both and .

3. Algorithm Design / Pseudocode

A_STAR (S, G)

1. Put S in OPEN list.
2. Set $g(S) = 0$.

3. Calculate $f(S) = g(S) + h(S)$.

4. Repeat until OPEN is empty:

a. Select node n with minimum $f(n)$.

b. If $n = G$, return the path.

c. Expand n and examine its neighbours.

d. Calculate:

$$g(\text{child}) = g(n) + \text{edge cost}$$

$$f(\text{child}) = g(\text{child}) + h(\text{child})$$

e. Add/update the child in OPEN.

5. Return the path with minimum cost.

4. Application of A* to the Given Road Network

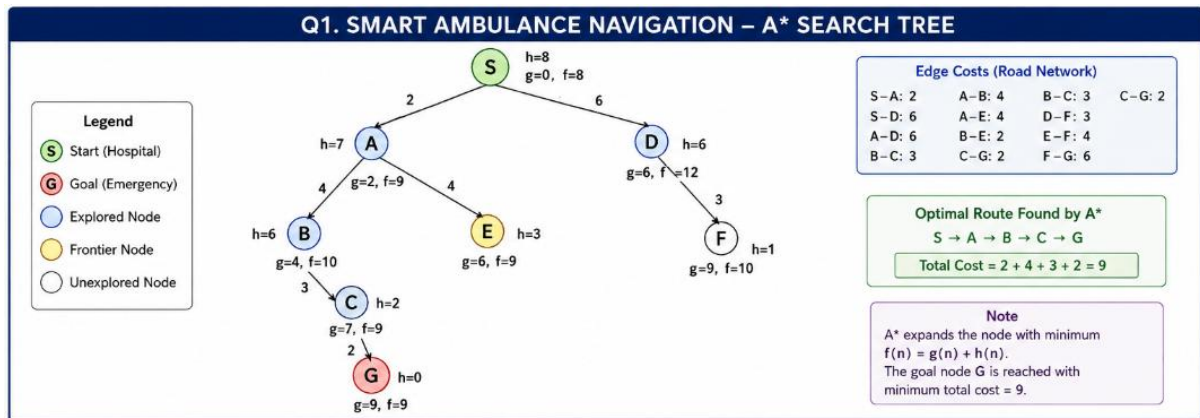
Starting from S:

Node	(g(n))	(h(n))	(f(n)=g+h)
S	0	8	8
A	2	7	9
D	6	6	12
E	6	3	9
B	4	6	10
C	7	2	9
G	9	0	9

A* first expands S. Its promising successors include A and D. A has , so A is selected. From A, the algorithm considers B, D and E. **E has** and B has .

The search subsequently reaches **B**, then **C**, and finally **G**.

5. Search Tree Representation



The selected optimal route is:

$$S \rightarrow A \rightarrow B \rightarrow C \rightarrow G$$

Cost:

$$2 + 2 + 3 + 2 = 9$$

Thus, the ambulance reaches the emergency location with a **total travel cost of 9**.

6. Complexity Analysis and Solution Efficiency

For A* Search, the worst-case time and space complexity can be approximately:

where b is the branching factor and d is the depth of the solution.

A good heuristic reduces unnecessary exploration and makes A* significantly more efficient than uninformed search.

7. Conclusion

A* Search is the most suitable approach because it considers both **actual travel cost** and **estimated remaining distance**. For the given road network, the algorithm selects:

$$S \rightarrow A \rightarrow B \rightarrow C \rightarrow G$$

with a minimum total cost of **9**. This provides the ambulance with an efficient route while demonstrating informed search and heuristic-based decision making.

Q2. Autonomous Warehouse Robot

1. Problem Analysis and Understanding

The warehouse robot must move from the **Charging Station (S)** to the **Target Shelf (G)** through a grid environment. The important challenge is that the robot **does not initially know all obstacles**. Some cells are marked as unknown (?), and an obstacle becomes known only when the robot reaches or senses a nearby location.

Therefore, a normal offline search that assumes the complete map is known is unsuitable. The robot requires an **online intelligent search strategy** that repeatedly:

1. Observes the environment.
2. Updates its map.
3. Selects the best currently known action.
4. Moves toward the goal.
5. Replans whenever a new obstacle is discovered.

2. AI Algorithm Selection & Justification

Selected Approach: Online Informed Search using A*-style replanning

The robot can use a Manhattan-distance heuristic:

$$h(n) = |x_n - x_G| + |y_n - y_G|$$

and evaluate a cell using:

$$f(n) = g(n) + h(n)$$

Initially, unknown cells are treated as **potentially traversable**, but the robot continuously checks nearby cells. If an obstacle is detected, that cell is marked blocked, and the path is calculated again.

This is better than a fixed offline plan because the environment is only **partially known**.

3. Algorithm Design / Pseudocode

ONLINE_ROBOT_SEARCH (S, G)

1. Start at S.

2. Mark the current position as known.
3. Sense nearby cells.
4. Update the internal map.
5. Use A* to find the best path using known information.
6. Move one safe step along the selected path.
7. Sense the new surroundings.
8. If a new obstacle is discovered:

Mark the cell as BLOCKED.

Recalculate the path.

9. Repeat Steps 5–8 until G is reached.
10. Stop when the target shelf is reached.

4. Application to the Given Grid

The robot starts at **S** in the upper-left region, and the target **G** is at the lower-right region.

Initially:

```

S ? ?? ?? ?
? ? ? # ? ? ?
? ? ? ? # ? #
? # ? ? ? # ?
? ? # ? ? ? G

```

S = Charging Station

G = Goal

= Obstacle

? = Unknown cell

The robot does **not assume that every ? cell is permanently free.**

For example, suppose the robot chooses a route toward the goal and discovers an obstacle in a previously unknown cell:

Before discovery:

$S \rightarrow \rightarrow \rightarrow \rightarrow \rightarrow G$

?

After sensing:

$S \rightarrow \rightarrow \rightarrow X$

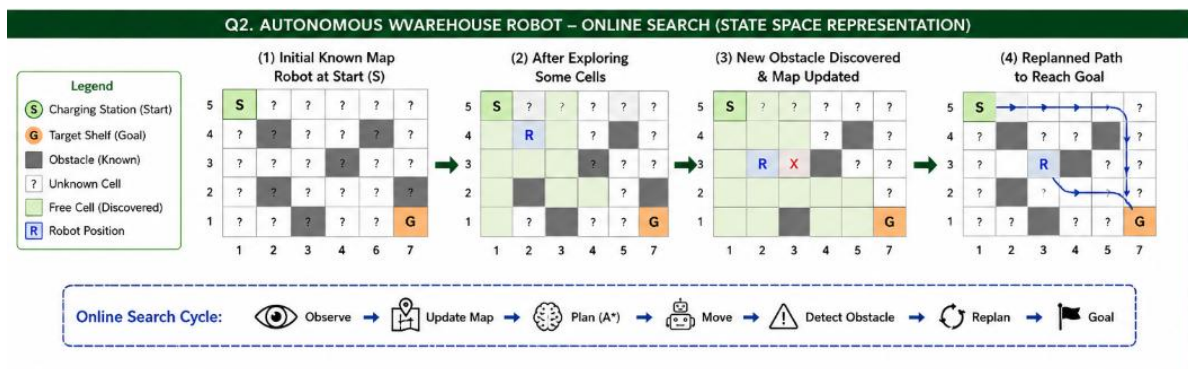
← Newly discovered obstacle

The robot immediately updates its internal map:

? → BLOCKED

It then performs another informed search and selects an alternative safe route.

5. Online State-Space Representation



This demonstrates the main principle of an online search agent: the robot does not create the entire solution once and blindly follow it. Instead, it repeatedly perceives, plans, acts and replans.

6. Comparison with Offline Search

Feature	Online Search	Offline Search
Initial map	Partially known	Completely known
Obstacle discovery	During movement	Before movement
Replanning	Required	Usually, unnecessary
Adaptability	High	Low

Suitable for this problem	Yes	No
---------------------------	-----	----

For the warehouse problem, online search is superior because unexpected obstacles can appear during navigation.

7. Complexity Analysis and Solution Efficiency

For A*-style search, the worst-case complexity is approximately:

$$O(bd)$$

where b is the number of possible movements from a cell and d is the search depth.

Online search may perform multiple replanning operations, increasing computational cost. However, it provides an important advantage: the robot can safely adapt to an unknown environment instead of following an invalid precomputed route.

8. Conclusion

An online informed search agent with A*-style replanning is appropriate for the autonomous warehouse robot. The robot observes its surroundings, updates its map, detects newly discovered obstacles and recalculates the safest route.

Thus:

Observe → Plan → Move → Detect → Update → Replan → Goal

Q3. University Examination Scheduling

1. Problem Analysis and Understanding

A university must schedule six examinations (E1–E6) into four time slots T1–T4. Exams having common students cannot occur simultaneously. The constraints are:

E1–E2, E1–E3, E2–E4, E2–E5, E3–E5, E4–E5.

Student counts are: E1=100, E2=80, E3=70, E4=60, E5=90, E6=50.

Hall capacities are A=120, B=90, C=60.

This is a Constraint Satisfaction Problem (CSP) where each exam is a variable, its domain is the four time slots, and the conflict/capacity rules are constraints.

2. AI Algorithm Selection & Justification

Selected: Backtracking CSP + Minimum Remaining Values (MRV) + Forward Checking.

- Backtracking: tries assignments and reverses an assignment when a constraint is violated.
- MRV: selects the most constrained examination first.
- Forward Checking: removes an assigned slot from the domains of conflicting examinations.

This combination reduces unnecessary search and quickly detects impossible assignments.

3. Algorithm Design / Pseudocode

CSP_SCHEDULE ()

1. Set Domain (E_i) = {T1, T2, T3, T4}.
2. Select an unassigned exam using MRV.
3. Select a valid time slot.
4. Check student-conflict constraints.
5. Check available hall capacity.
6. Assign the slot and hall.

7. Apply Forward Checking to conflicting exams.

8. If any domain becomes empty:

Backtrack.

9. If all exams are assigned:

Return the timetable.

10. Otherwise repeat from Step 2.

4. Application of AI Concepts

A valid schedule is:

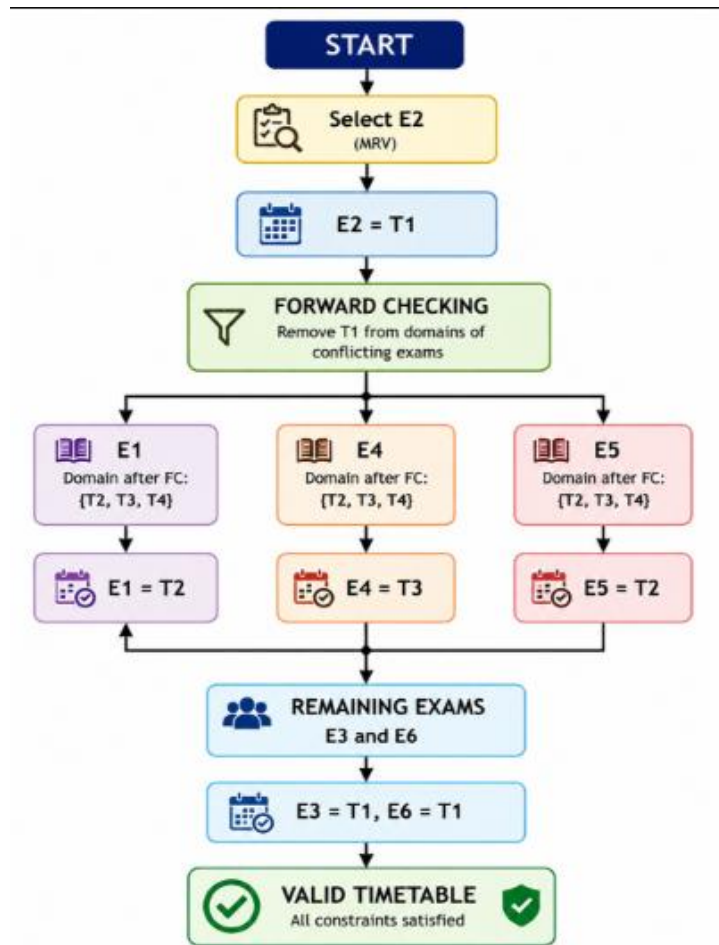
Time	Hall A	Hall B	Hall C
T1	E3 (70)	E2 (80)	E6 (50)
T2	E1 (100)	E5 (90)	—
T3	—	E4 (60)	—

Conflict checking:

- E1 and E2 → different slots
- E1 and E3 → different slots
- E2 and E4 → different slots
- E2 and E5 → different slots
- E3 and E5 → different slots
- E4 and E5 → different slots

All hall capacities are also satisfied.

5. State-Space Representation



6. Complexity Analysis & Efficiency

For examinations and possible slots, ordinary backtracking has worst-case complexity:

$$O(dn)$$

Here:

$$46 \times 4096$$

possible basic assignments exist. **MRV and Forward Checking** reduce this search considerably by selecting constrained exams first and eliminating invalid slots early.

7. Conclusion

CSP with **MRV and Forward Checking** produces a valid examination timetable while satisfying both **student-conflict and hall-capacity constraints**. It is more efficient than blind trial-and-error because invalid choices are eliminated early.

CSP + MRV + Forward Checking

Q4. Game-Playing AI

1. Problem Analysis and Understanding

The given game tree represents a two-player competitive game. The root is a **MAX** node, followed by **MIN** nodes and then MAX nodes containing terminal scores.

- **MAX:** AI tries to maximize the score.
- **MIN:** opponent tries to minimize the score.

The task is to determine the best move using **Minimax**, then apply **Alpha-Beta Pruning** to avoid unnecessary evaluation.

2. AI Algorithm Selection & Justification

Selected: Minimax with Alpha-Beta Pruning.

Minimax is appropriate for a two-player, zero-sum game because it assumes both players make optimal decisions.

MAX=maximum child value

MIN=minimum child value

Alpha-Beta pruning improves efficiency by removing branches that cannot change the final decision.

3. Algorithm Design / Pseudocode

MINIMAX (node)

IF node is terminal:

 RETURN utility(node)

IF node is MAX:

 RETURN maximum MINIMAX (child)

IF node is MIN:

 RETURN minimum MINIMAX (child)

Alpha-Beta:

ALPHA_BETA (node, α , β)

IF terminal:

 RETURN value

IF MAX:

 for each child:

$\alpha = \max (\alpha, \text{ALPHA_BETA (child, } \alpha, \beta))$

 IF $\alpha \geq \beta$: PRUNE

 return α

IF MIN:

 for each child:

$\beta = \min (\beta, \text{ALPHA_BETA (child, } \alpha, \beta))$

 IF $\beta \leq \alpha$: PRUNE

 return β

4. Application of Minimax

Evaluating the supplied tree from left to right:

Left MIN branch

MAX (3,5,6) =6

MAX (9,1,2) =9

$$\text{MAX } (0, -1, 4) = 4$$

Therefore:

$$\text{MIN } (6, 9, 4) = 4$$

Middle MIN branch

$$\text{MAX } (7, 5, 8) = 8$$

$$\text{MAX } (6, 2, 3) = 6$$

$$\text{MAX } (4, 0, -2) = 4$$

Therefore:

$$\text{MIN } (8, 6, 4) = 4$$

Right MIN branch

$$\text{MAX } (1, 2, 3) = 3$$

$$\text{MAX } (5, -1, 2) = 5$$

$$\text{MAX } (6, 4, 7) = 7$$

Therefore:

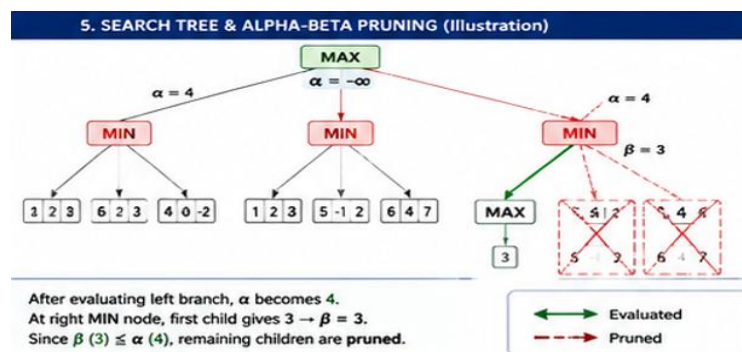
$$\text{MIN } (3, 5, 7) = 3$$

At the root:

$$\text{MAX } (4, 4, 3) = 4$$

Thus, the AI should choose **either the left or middle branch**, both giving a guaranteed score of 4.

5. Search Tree Representation



6. Alpha-Beta Pruning

After evaluating the left branch, MAX obtains:

$$\alpha=4$$

At the right MIN node, the first child produces:

$$3$$

Therefore:

$$\beta=3$$

Since:

$$\beta \leq \alpha$$

the remaining two right-side branches do not need to be evaluated.

Thus, the branches containing:

(5, -1,2) And (6,4,7) can be **pruned**.

7. Complexity Analysis & Efficiency

For branching factor and depth:

Minimax: $O(bd)$

Alpha-Beta has the same worst case, but with good move ordering can approach: $O(bd/2)$

Therefore, Alpha-Beta allows the AI to search deeper while evaluating fewer game states.

Conclusion

Minimax + Alpha-Beta Pruning provides the optimal decision for the supplied game tree.

The best root value is:

Q5. Delivery Route Optimization

1. Problem Analysis and Understanding

A logistics company wants to minimize the total distance travelled by a delivery vehicle visiting customers **A, B, C and E**, starting and ending at **D**. Several candidate routes are given.

The objective is to find a shorter route using **Local Search / Hill Climbing**. The algorithm evaluates neighbouring routes and moves to a better solution until no improving neighbour is available.

The given initial routes include:

Route	Cost
S1: D→A→C→B→E→D	26 km
S2: D→B→A→C→E→D	30 km
S3: D→A→B→E→C→D	24 km
S4: D→E→C→B→A→D	27 km
S5: D→A→E→B→C→D	23 km
S6: D→C→B→A→E→D	29 km

2. AI Algorithm Selection & Justification

Selected Algorithm: Hill Climbing (Local Search)

Hill Climbing is suitable because the objective is to **minimize route cost** by repeatedly replacing the current solution with a better neighboring solution.

Advantages:

- Simple and memory efficient.
- Quickly improves a given route.
- Does not need to store the complete search tree.
- Suitable for route optimization problems.

Its main limitation is that it may become trapped in a **local optimum**.

3. Algorithm Design / Pseudocode

HILL_CLIMBING(initial_route)

1. $\text{current} \leftarrow \text{initial_route}$

2. Calculate cost(current)

3. Repeat:

 Generate neighboring routes

 Calculate cost of each neighbor

$\text{best} \leftarrow \text{neighbor with minimum cost}$

 IF $\text{cost}(\text{best}) < \text{cost}(\text{current})$:

$\text{current} \leftarrow \text{best}$

 ELSE:

 STOP

4. Return current route and its cost

The algorithm therefore follows:

Current Route → Generate Neighbors → Select Best → Improve → Repeat

4. Application of Hill Climbing

Among the supplied initial solutions, **S5 has the lowest given cost of 23 km:**

$S5 = D \rightarrow A \rightarrow E \rightarrow B \rightarrow C \rightarrow D$

$\text{Cost}(S5) = 23 \text{ km}$

The question's supplied information states that **the best neighbour of S5 has cost 22 km.**

Therefore, Hill Climbing makes the improvement:

$23 \rightarrow 22$

Since the neighbouring solutions of this new solution have costs:

$\geq 22 \text{ km}$

no further improvement is possible.

Thus, according to the given problem data:

Final local-search cost = 22 km

Note: The provided question image gives the 22-km neighbour's cost but does not explicitly show its route sequence, so the route should be recorded as the selected 22-km neighbour rather than inventing a sequence.

5. Search / Local-State Representation



6. Complexity Analysis & Solution Efficiency

If there are customers and neighbours are examined at each iteration: $O(k)$

per iteration, with total complexity approximately: $O(kI)$

where I is the number of improvement iterations.

Hill Climbing uses relatively little memory because it normally stores only the current solution and its neighbours.

7. Global Optimum, Limitations & Improvement

The 22-km solution is a local optimum, because none of its immediate neighbors improves the cost.

It cannot automatically be guaranteed as a global optimum unless all possible routes are examined.

A local optimum can be escaped using:

- Random Restart Hill Climbing
- Simulated Annealing
- Allowing occasional non-improving moves
- Starting from multiple different routes

Random Restart is particularly useful because the algorithm can begin from several different initial routes and retain the best solution found.

Conclusion

Hill Climbing successfully improves the supplied solution from:

23 km → 22 km

and stops when no better neighbouring route exists. Therefore, 22 km is the final local optimum according to the supplied problem data, but additional search such as Random Restart or Simulated Annealing is required to increase confidence that the solution is globally optimal.

EVALUATION RUBRICS

Criteria	Weightage (%)	Excellent (Full Marks)	Good	Average	Poor
CSP Problem Formulation	20%	4% – Clearly defines all variables, domains, and problem objectives correctly.	3% – Defines most variables and domains with minor errors.	2% – Partial formulation with some missing components.	0–1% – Incorrect or incomplete formulation.
Constraint Identification & Representation	20%	4% – Accurately identifies all student-conflict and hall-capacity constraints and represents them correctly.	3% – Identifies most constraints with minor errors.	2% – Identifies some constraints but misses important relationships.	0–1% – Constraints are largely incorrect or missing.
Application of Backtracking, MRV & Forward Checking	30%	6% – Correctly applies Backtracking, MRV, and Forward Checking with clear and logical steps.	4.5% – Applies the techniques correctly with minor mistakes.	3% – Partial application with unclear or inconsistent steps.	0–1.5% – Incorrect or inappropriate application of CSP techniques.
Step-by-Step Execution & Domain Updates	20%	4% – Provides a complete, logical execution trace and correctly updates domains after every assignment.	3% – Mostly correct execution with minor gaps.	2% – Provides limited steps with some incorrect domain updates.	0–1% – Incomplete or incorrect execution.

Final Timetable, Accuracy & Justification	10%	2% – Produces a valid conflict-free timetable satisfying all constraints and capacity requirements with clear justification.	1.5% – Valid timetable with minor errors in justification.	1% – Partially valid timetable with some constraint violations.	0–0.5% – Invalid or missing timetable.
--	-----	--	--	--	---